

- **Objective / Aim**

To design and implement a in Java using , demonstrating object-oriented programming concepts such as attributes, methods, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, and fund transfers.

Algorithm / Logic Description

Create a base class named Account.

Define the following attributes in the Account class:

- Account number
- Account holder name
- Balance
- Account type

Define a constructor to initialize the account details.

Define the following methods in the Account class:

- deposit() to add money to the account.
- withdraw() to withdraw money when sufficient balance is available.
- transfer() to transfer money from one account to another.
- displayAccountDetails() to display account information.

Create a SavingsAccount class that inherits from Account.

Add interestRate as an additional attribute in SavingsAccount.

Define calculateInterest() to calculate interest based on the account balance.

Create a CurrentAccount class that inherits from Account.

Add overdraftLimit as an additional attribute in CurrentAccount.

Override the withdraw() method in CurrentAccount to allow withdrawal up to the permitted overdraft limit.

Create objects for SavingsAccount and CurrentAccount.

Perform deposits, withdrawals, interest calculation, and fund transfer operations.

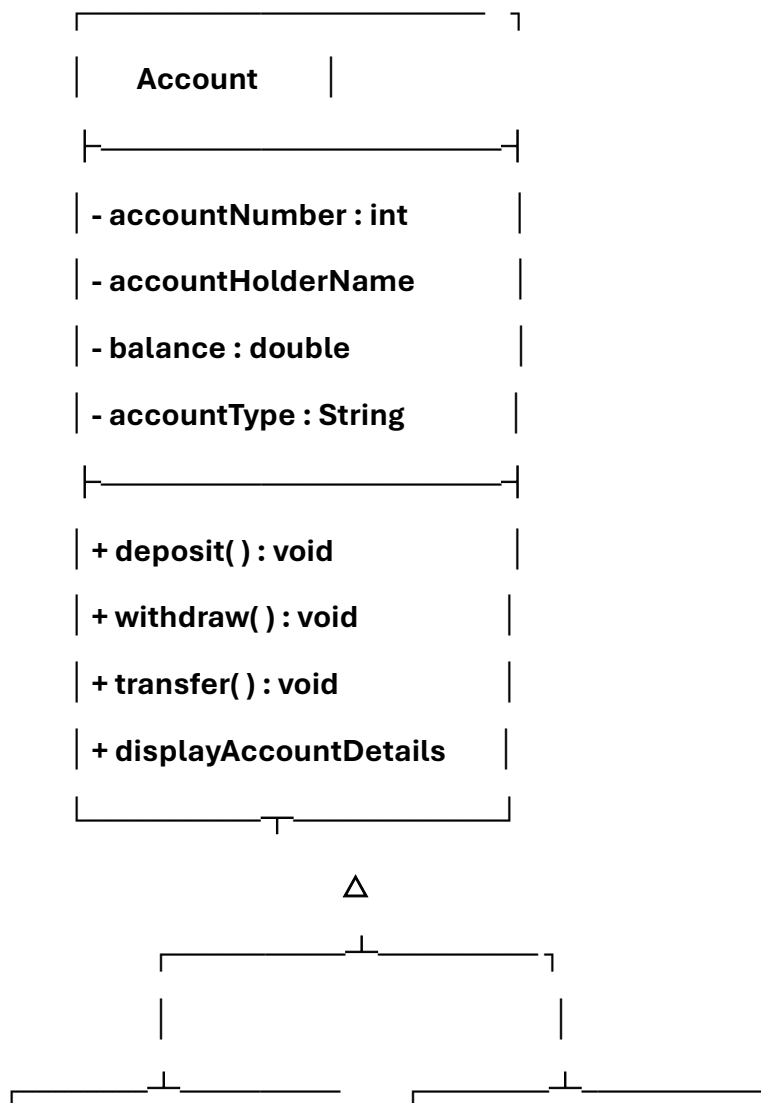
Display the account details before and after the transactions.

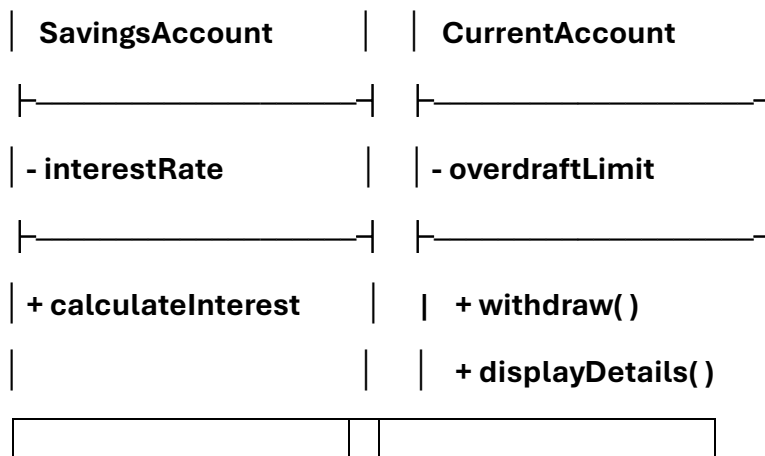
Display the final balance of both accounts.

NOTE:

SavingsAccount and CurrentAccount inherit from the Account class.

UML Class Diagram:





Code:

```
class Account {
    // Attributes
    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    // Constructor
    Account(int accountNumber, String accountHolderName,double balance, String
accountType) {

        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit method
```

```
void deposit(double amount) {  
    if (amount > 0) {  
        balance = balance + amount;  
        System.out.println("Deposited: " + amount);  
    } else {  
        System.out.println("Invalid deposit amount");  
    }  
}
```

// Withdraw method

```
void withdraw(double amount) {  
    if (amount > 0 && amount <= balance) {  
        balance = balance - amount;  
        System.out.println("Withdraw: " + amount);  
    } else {  
        System.out.println("Insufficient balance");  
    }  
}
```

// Transfer method

```
void transfer(Account receiver, double amount) {  
    if (amount > 0 && amount <= balance) {  
        balance = balance - amount;  
        receiver.balance = receiver.balance + amount;  
  
        System.out.println("Transferred: " + amount);  
        System.out.println("From Account: " + accountNumber);  
        System.out.println("To Account: " + receiver.accountNumber);  
    }  
}
```

```

    } else {
        System.out.println("Transfer failed - Insufficient balance");
    }
}

// Display account details
void displayAccountDetails() {
    System.out.println("\n--- Account Details ---");
    System.out.println("Account Number : " + accountNumber);
    System.out.println("Account Holder : " + accountHolderName);
    System.out.println("Account Type : " + accountType);
    System.out.println("Balance : " + balance);
}
}

// SavingsAccount inherits Account
class SavingsAccount extends Account {

    double interestRate;

    // Constructor
    SavingsAccount(int accountNumber, String accountHolderName, double
balance, double interestRate) {

        super(accountNumber, accountHolderName, balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculate interest

```

```
void calculateInterest() {  
    double interest = (balance * interestRate) / 100;  
    balance = balance + interest;  
  
    System.out.println("Interest Rate   : " + interestRate + "%");  
    System.out.println("Interest Earned  : " + interest);  
}  
}
```

// CurrentAccount inherits Account

```
class CurrentAccount extends Account {
```

```
    double overdraftLimit;
```

// Constructor

```
    CurrentAccount(int accountNumber, String accountHolderName, double  
balance, double overdraftLimit) {
```

```
        super(accountNumber, accountHolderName, balance, "Current");
```

```
        this.overdraftLimit = overdraftLimit;
```

```
}
```

// Method overriding

```
@Override
```

```
void withdraw(double amount) {
```

```
    if (amount > 0 && amount <= balance + overdraftLimit) {
```

```
        balance = balance - amount;
```

```

        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Withdrawal exceeds overdraft limit");
    }
}

// Display current account details
@Override
void displayAccountDetails() {
    super.displayAccountDetails();
    System.out.println("Overdraft Limit : " + overdraftLimit);
}
}

public class BankMangementSystem {

    public static void main(String[] args) {

        SavingsAccount sa = new SavingsAccount(101, "Sahithi", 10000, 5);

        // Creating CurrentAccount object
        CurrentAccount ca = new CurrentAccount(102, "Ashmitha", 5000, 3000);

        // Display details before transactions
        System.out.println(" BEFORE TRANSACTIONS");

        sa.displayAccountDetails();
        ca.displayAccountDetails();
    }
}

```

```
// Savings account operations

System.out.println("\nSAVINGS ACCOUNT OPERATIONS");


sa.deposit(2000);

sa.withdraw(1000);

sa.calculateInterest();


// Current account operations

System.out.println("\nCURRENT ACCOUNT OPERATIONS");


ca.deposit(3000);

ca.withdraw(9000);


// Transfer money from Savings to Current

System.out.println("\nFUND TRANSFER");


sa.transfer(ca, 2000);


// Display details after transactions

System.out.println("\n AFTER TRANSACTIONS");


sa.displayAccountDetails();

ca.displayAccountDetails();


// Final balances

System.out.println("\n FINAL BALANCES");
```



```
System.out.println("Savings Account Final Balance : "+ sa.balance);
```

```
System.out.println("Current Account Final Balance : "+ ca.balance);
```

```
}
```

```
}
```

Output:

```
PS C:\Users\Sahithi\pyla\Desktop\Sahithi_java> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessage' 'C:\Users\Sahithi\pyla\Desktop\Sahithi_java\src\main\java\com\redhat\java\jdt_ws\Sahithi_java_e658ba66\bin' 'BankManagementSystem'
BEFORE TRANSACTIONS

--- Account Details ---
Account Number : 101
Account Holder : Sahithi
Account Type : Savings
Balance : 10000.0

--- Account Details ---
Account Number : 102
Account Holder : Ashmitha
Account Type : Current
Balance : 5000.0
Overdraft Limit : 3000.0

SAVINGS ACCOUNT OPERATIONS
Deposited: 2000.0
Withdraw: 1000.0
Interest Rate : 5.0%
Interest Earned : 550.0

CURRENT ACCOUNT OPERATIONS
Deposited: 3000.0
Withdrawn: 9000.0

FUND TRANSFER
Transferred: 2000.0
From Account: 101
To Account: 102

AFTER TRANSACTIONS

--- Account Details ---
Account Number : 101
Account Holder : Sahithi
Account Type : Savings
Balance : 9550.0
```

Result:

The **Bank Account Management System** was successfully implemented in Java using **classes and objects**. The program demonstrates the creation of SavingsAccount and CurrentAccount objects and performs deposits, withdrawals, interest calculation, and fund transfers successfully